

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»**

(Университет ИТМО)

Факультет Прикладной информатики

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии

**ОТЧЕТ ПО Лабораторной работе 4 Проектирование и реализация баз
данных**

**Тема: «Запросы на выборку и модификацию данных. Представления. Работа
с индексами.»**

Обучающийся: Ипатова Ульяна Юрьевна К3239

Вариант 10

Санкт-Петербург 2026

voditel.last_name,

```

reis."bus_ID"
FROM bus_station_sch.reis
JOIN bus_station_sch.ekipazh
ON reis."flight_ID" = ekipazh."flight_ID"
JOIN bus_station_sch.voditel
ON ekipazh."driver_ID" = voditel."driver_ID"
WHERE reis.departure_date = CURRENT_DATE
AND reis.departure_time < TIME '12:00'
AND ekipazh.medical_check_up_status = 'P'
ORDER BY voditel.last_name, reis."bus_ID";

```

The screenshot shows a PostgreSQL query editor interface. The top bar indicates the connection is to 'bus_station/postgres@PostgreSQL 15'. Below the toolbar, the 'Query' tab is active, displaying the following SQL query:

```

1  SELECT DISTINCT
2      voditel.last_name,
3      reis."bus_ID",
4      reis.departure_time
5  FROM bus_station_sch.reis
6  JOIN bus_station_sch.ekipazh
7      ON reis."flight_ID" = ekipazh."flight_ID"
8  JOIN bus_station_sch.voditel
9      ON ekipazh."driver_ID" = voditel."driver_ID"
10 WHERE reis.departure_date = CURRENT_DATE
11      AND reis.departure_time < TIME '12:00'
12      AND ekipazh.medical_check_up_status = 'P'
13 ORDER BY voditel.last_name, reis."bus_ID";

```

Below the query editor, the 'Data Output' tab is active, showing the results of the query. The results are displayed in a table with 4 columns: last_name, bus_ID, and departure_time. The table contains 4 rows of data.

	last_name character varying (30)	bus_ID character (10)	departure_time time without time zone
1	Иванов	BUS001	08:30:00
2	Кузнецов	BUS002	10:45:00
3	Петров	BUS001	08:30:00
4	Сидоров	BUS002	10:45:00

Рассчитать выручку от продажи билетов за прошедший день.

```

SELECT
SUM(bilet.ticket_price) AS revenue_previous_day
FROM bus_station_sch.bilet

```

```

JOIN bus_station_sch.reis
ON билет."flight_ID" = reis."flight_ID"
WHERE reis.departure_date = CURRENT_DATE - 1
AND билет.ticket_status = 'PAID';

```

The screenshot shows a PostgreSQL query editor interface. The title bar indicates the connection is to 'bus_station/postgres@PostgreSQL 15'. The query editor contains the following SQL query:

```

1  SELECT
2      SUM(билет.ticket_price) AS revenue_previous_day
3  FROM bus_station_sch.билет
4  JOIN bus_station_sch.reis
5      ON билет."flight_ID" = reis."flight_ID"
6  WHERE reis.departure_date = CURRENT_DATE - 1
7      AND билет.ticket_status = 'PAID';

```

Below the query editor, the 'Data Output' tab is active, showing the results of the query. The results are displayed in a table with two columns: 'revenue_previous_day' (numeric) and a single row with the value '22000.00'.

	revenue_previous_day numeric
1	22000.00

Вывести список водителей, которые не выполнили ни одного рейса за прошедшую неделю.

```

SELECT DISTINCT
voditel."driver_ID",
voditel.last_name,
voditel.name,
voditel.middle_name
FROM bus_station_sch.voditel
WHERE voditel."driver_ID" NOT IN
(
    SELECT ekipazh."driver_ID"

```

```

FROM bus_station_sch.ekipazh
JOIN bus_station_sch.reis
    ON ekipazh."flight_ID" = reis."flight_ID"
WHERE ekipazh.medical_check_up_status = 'P'
    AND reis.departure_date BETWEEN CURRENT_DATE - 7 AND CURRENT_DATE - 1
)
ORDER BY voditel.last_name, voditel.name, voditel.middle_name;

```

bus_station/postgres@PostgreSQL 15

Query Query History

```

1  SELECT DISTINCT
2      voditel."driver_ID",
3      voditel.last_name,
4      voditel.name,
5      voditel.middle_name
6  FROM bus_station_sch.voditel
7  WHERE voditel."driver_ID" NOT IN
8      (
9      SELECT ekipazh."driver_ID"
10     FROM bus_station_sch.ekipazh
11     JOIN bus_station_sch.reis
12         ON ekipazh."flight_ID" = reis."flight_ID"
13     WHERE ekipazh.medical_check_up_status = 'P'
14         AND reis.departure_date BETWEEN CURRENT_DATE - 7 AND CURRENT_DATE - 1
15     )
16  ORDER BY voditel.last_name, voditel.name, voditel.middle_name;

```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1

	driver_ID [PK] integer	last_name character varying (30)	name character varying (30)	middle_name character varying (30)
1	10	Волков	Никита	Алексеевич
2	9	Морозов	Константин	Викторович

Вывести сумму убытков из-за непроданных мест в автобусе за прошедшую неделю.

bus_station/postgres@PostgreSQL 15

Query Query History

```
1 SELECT
2     SUM((m.place_number - t.sold_places) * t.min_price) AS lost_profit_last_week
3 FROM
4     (
5         SELECT
6             b."flight_ID",
7             COUNT(*) AS sold_places,
8             MIN(b.ticket_price) AS min_price
9         FROM bus_station_sch.bilet b
10        JOIN bus_station_sch.reis r
11         ON b."flight_ID" = r."flight_ID"
12        WHERE b.ticket_status = 'PAID'
13              AND b.place_number IS NOT NULL
14              AND r.departure_date BETWEEN CURRENT_DATE - 7 AND CURRENT_DATE - 1
15        GROUP BY b."flight_ID"
16    ) t
17 JOIN bus_station_sch.reis r
18     ON t."flight_ID" = r."flight_ID"
19 JOIN bus_station_sch.avtobus a
20     ON r."bus_ID" = a."bus_ID"
21 JOIN bus_station_sch.model_avtobusa m
22     ON a."model_ID" = m."model_ID";
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1

	lost_profit_last_week
	numeric
1	909000.00

Найти самый популярный маршрут за прошедший месяц (маршрут с максимальной выручкой)

```
SELECT *
FROM
(
    SELECT
        p.destination_name AS route_name,
        SUM(b.ticket_price) AS total_revenue
    FROM bus_station_sch.reis
    JOIN bus_station_sch.punkt_naznacheniya
        ON reis."destination_ID" = punkt_naznacheniya."destination_ID"
    JOIN bus_station_sch.bilet
        ON reis."flight_ID" = bilet."flight_ID"
    WHERE reis.departure_date BETWEEN CURRENT_DATE - 30 AND CURRENT_DATE - 1
          AND bilet.ticket_status = 'PAID'
    GROUP BY punkt_naznacheniya.destination_name
) route_sum
WHERE total_revenue =
(
    SELECT MAX(total_revenue)
```

```

FROM
(
    SELECT
        punkt_naznacheniya.destination_name AS route_name,
        SUM(bilet.ticket_price) AS total_revenue
    FROM bus_station_sch.reis
    JOIN bus_station_sch.punkt_naznacheniya
        ON reis."destination_ID" = punkt_naznacheniya."destination_ID"
    JOIN bus_station_sch.bilet
        ON reis."flight_ID" = bilet."flight_ID"
    WHERE reis.departure_date BETWEEN CURRENT_DATE - 30 AND CURRENT_DATE
- 1
        AND bilet.ticket_status = 'PAID'
    GROUP BY punkt_naznacheniya.destination_name
) max_table
);

```

bus_station/postgres@PostgreSQL 15

Query Query History

```

1  SELECT *
2  FROM
3  (
4      SELECT
5          p.destination_name AS route_name,
6          SUM(b.ticket_price) AS total_revenue
7      FROM bus_station_sch.reis r
8      JOIN bus_station_sch.punkt_naznacheniya p
9          ON r."destination_ID" = p."destination_ID"
10     JOIN bus_station_sch.bilet b
11         ON r."flight_ID" = b."flight_ID"
12     WHERE r.departure_date BETWEEN CURRENT_DATE - 30 AND CURRENT_DATE - 1
13         AND b.ticket_status = 'PAID'
14     GROUP BY p.destination_name
15 ) route_sum
16 WHERE total_revenue =
17 (
18     SELECT MAX(total_revenue)
19     FROM
20     (
21         SELECT
22             p.destination_name AS route_name,
23             SUM(b.ticket_price) AS total_revenue
24         FROM bus_station_sch.reis r
25         JOIN bus_station_sch.punkt_naznacheniya p
26             ON r."destination_ID" = p."destination_ID"
27         JOIN bus_station_sch.bilet b
28             ON r."flight_ID" = b."flight_ID"

```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1

	route_name character varying (100)	total_revenue numeric
1	Москва	26400.00

Проверка запроса:

The screenshot shows a PostgreSQL query editor interface. The connection is 'bus_station/postgres@PostgreSQL 15'. The query is as follows:

```
1 SELECT
2     p.destination_name AS route_name,
3     SUM(b.ticket_price) AS total_revenue
4 FROM bus_station_sch.reis r
5 JOIN bus_station_sch.punkt_naznacheniya p
6     ON r."destination_ID" = p."destination_ID"
7 JOIN bus_station_sch.bilet b
8     ON r."flight_ID" = b."flight_ID"
9 WHERE r.departure_date BETWEEN CURRENT_DATE - 30 AND CURRENT_DATE - 1
10 AND b.ticket_status = 'PAID'
11 GROUP BY p.destination_name
12 ORDER BY total_revenue DESC;
```

Below the query, the 'Data Output' tab shows the results of the query:

	route_name character varying (100)	total_revenue numeric
1	Москва	26400.00
2	Псков	15000.00
3	Великий Новгород	14400.00
4	Тверь	8400.00
5	Валдай	6800.00
6	Боровичи	1800.00

Вывести тип автобуса, который используется на всех рейсах.

```
SELECT *
FROM
(
    SELECT
        model_avtobusa.bus_type,
        COUNT(DISTINCT reis."flight_ID") AS flight_count
    FROM bus_station_sch.reis
    JOIN bus_station_sch.avtobus
        ON reis."bus_ID" = avtobus."bus_ID"
    JOIN bus_station_sch.model_avtobusa
```



```

        ON avtobus."model_ID" = model_avtobusa."model_ID"
    GROUP BY model_avtobusa.bus_type
) t
WHERE flight_count =
(
    SELECT COUNT(*)
    FROM bus_station_sch.reis
);

```

The screenshot shows a PostgreSQL query editor interface. The top bar indicates the connection is to 'bus_station/postgres@PostgreSQL 15'. Below the toolbar, the 'Query' tab is active, displaying a SQL query. The query is a complex join and aggregation statement. The 'Data Output' tab at the bottom shows the results of the query, with columns 'bus_type' and 'flight_count'.

Query:

```

1  SELECT *
2  FROM
3  (
4      SELECT
5          m.bus_type,
6          COUNT(DISTINCT r."flight_ID") AS flight_count
7      FROM bus_station_sch.reis r
8      JOIN bus_station_sch.avtobus a
9          ON r."bus_ID" = a."bus_ID"
10     JOIN bus_station_sch.model_avtobusa m
11         ON a."model_ID" = m."model_ID"
12     GROUP BY m.bus_type
13 ) t
14 WHERE flight_count =
15 (
16     SELECT COUNT(*)
17     FROM bus_station_sch.reis
18 );

```

Data Output:

bus_type	flight_count
character varying (30)	bigint

Проверка:

```

SELECT
m.bus_type,
COUNT(DISTINCT r."flight_ID") AS flight_count
FROM bus_station_sch.reis r
JOIN bus_station_sch.avtobus a
ON r."bus_ID" = a."bus_ID"
JOIN bus_station_sch.model_avtobusa m

```

```
ON a."model_ID" = m."model_ID"
GROUP BY m.bus_type;
```

The screenshot shows a PostgreSQL query editor interface. The title bar indicates the connection is to 'bus_station/postgres@PostgreSQL 15'. The query editor contains the following SQL code:

```
1 SELECT
2     m.bus_type,
3     COUNT(DISTINCT r."flight_ID") AS flight_count
4 FROM bus_station_sch.reis r
5 JOIN bus_station_sch.avtobus a
6     ON r."bus_ID" = a."bus_ID"
7 JOIN bus_station_sch.model_avtobusa m
8     ON a."model_ID" = m."model_ID"
9 GROUP BY m.bus_type;
```

Below the query editor, the 'Data Output' tab is active, displaying the results of the query. The results are shown in a table with two columns: 'bus_type' (character varying (30)) and 'flight_count' (bigint). The table contains three rows of data:

	bus_type	flight_count
1	городской	4
2	междугородний	16
3	туристический	4

Всего в таблице рейсов: 24

Вывести данные водителя, который провел максимальное время в пути за прошедшую неделю.

```
SELECT
    t."driver_ID",
    t.last_name,
    t.name,
    t.middle_name,
    t.total_time
FROM
    (
        SELECT
            v."driver_ID",
            v.last_name,
```

```

        v.name,
        v.middle_name,
        SUM(f.max_travel_time) AS total_time
FROM bus_station_sch.voditel v
JOIN bus_station_sch.ekipazh e
    ON v."driver_ID" = e."driver_ID"
JOIN bus_station_sch.reis r
    ON e."flight_ID" = r."flight_ID"
JOIN
(
    SELECT
        "flight_ID",
        MAX(travel_time) AS max_travel_time
    FROM bus_station_sch.promezhutochnye_ostanovki
    GROUP BY "flight_ID"
) f
    ON e."flight_ID" = f."flight_ID"
WHERE e.medical_check_up_status = 'P'
    AND r.departure_date BETWEEN CURRENT_DATE - 7 AND CURRENT_DATE - 1
GROUP BY
    v."driver_ID",
    v.last_name,
    v.name,
    v.middle_name
) t
WHERE t.total_time =
(
    SELECT MAX(t2.total_time)
    FROM
    (
        SELECT
            v."driver_ID",
            SUM(f.max_travel_time) AS total_time
        FROM bus_station_sch.voditel v
        JOIN bus_station_sch.ekipazh e

```

```

        ON v."driver_ID" = e."driver_ID"
    JOIN bus_station_sch.reis r
        ON e."flight_ID" = r."flight_ID"
    JOIN
    (
        SELECT
            "flight_ID",
            MAX(travel_time) AS max_travel_time
        FROM bus_station_sch.promezhutochnye_ostanovki
        GROUP BY "flight_ID"
    ) f
        ON e."flight_ID" = f."flight_ID"
    WHERE e.medical_check_up_status = 'P'
        AND r.departure_date BETWEEN CURRENT_DATE - 7 AND CURRENT_DATE - 1
    GROUP BY v."driver_ID"
) t2
);

```

bus_station/postgres@PostgreSQL 15

Query Query History Scratch Pad x

```

1  SELECT
2      t."driver_ID",
3      t.last_name,
4      t.name,
5      t.middle_name,
6      t.total_time
7  FROM
8      (
9          SELECT

```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	driver_ID [PK] integer	last_name character varying (30)	name character varying (30)	middle_name character varying (30)	total_time interval
1	1	Иванов	Сергей	Петрович	24:45:00

Проверка:

```
SELECT
    v."driver_ID",
    v.last_name,
    v.name,
    v.middle_name,
    SUM(f.max_travel_time) AS total_time
FROM bus_station_sch.voditel v
JOIN bus_station_sch.ekipazh e
    ON v."driver_ID" = e."driver_ID"
JOIN bus_station_sch.reis r
    ON e."flight_ID" = r."flight_ID"
JOIN
(
    SELECT
        "flight_ID",
        MAX(travel_time) AS max_travel_time
    FROM bus_station_sch.promezhutochnye_ostanovki
    GROUP BY "flight_ID"
) f
    ON e."flight_ID" = f."flight_ID"
WHERE e.medical_check_up_status = 'P'
    AND r.departure_date BETWEEN CURRENT_DATE - 7 AND CURRENT_DATE - 1
GROUP BY v."driver_ID", v.last_name, v.name, v.middle_name
ORDER BY total_time DESC;
```

bus_station/postg... X bus_station/postg... X bus_station/postgres@PostgreSQL 15* X

bus_station/postgres@PostgreSQL 15

Query Query History Scratch Pad X

```

17 FROM bus_station_sch.promezhutochnye_ostanovi
18 GROUP BY "flight_ID"
19 ) f
20 ON e."flight_ID" = f."flight_ID"
21 WHERE e.medical_check_up_status = 'P'
22 AND r.departure_date BETWEEN CURRENT_DATE - 7 AND C
23 GROUP BY v."driver_ID", v.last_name, v.name, v.middle
24 ORDER BY total_time DESC;
25

```

Data Output Messages Notifications

Showing rows: 1 to 8 Page No: 1 of 1

	driver_ID [PK] integer	last_name character varying (30)	name character varying (30)	middle_name character varying (30)	total_time interval
1	1	Иванов	Сергей	Петрович	24:45:00
2	8	Фёдоров	Павел	Александрович	17:15:00
3	4	Кузнецов	Дмитрий	Андреевич	13:45:00
4	5	Васильев	Игорь	Сергеевич	13:20:00
5	3	Сидоров	Михаил	Олегович	10:55:00
6	2	Петров	Алексей	Николаевич	09:45:00
7	7	Смирнов	Евгений	Иванович	08:45:00
8	6	Попов	Роман	Владимирович	08:00:00

Видно, что действительно самый длинный рейс - это первая строчка.

2. Представления VIEW:

количество свободных мест на все рейсы на завтра;

```

CREATE VIEW bus_station_sch.vw_free_seats_tomorrow AS
SELECT
    r."flight_ID",
    p.destination_name AS route_name,
    r.departure_date,
    r.departure_time,
    m.place_number AS total_places,
    m.place_number - COUNT(b.place_number) AS free_places
FROM bus_station_sch.reis r
JOIN bus_station_sch.avtobus a
    ON r."bus_ID" = a."bus_ID"
JOIN bus_station_sch.model_avtobusa m
    ON a."model_ID" = m."model_ID"
JOIN bus_station_sch.punkt_naznacheniya p
    ON r."destination_ID" = p."destination_ID"
LEFT JOIN bus_station_sch.bilet b
    ON r."flight_ID" = b."flight_ID"

```

```

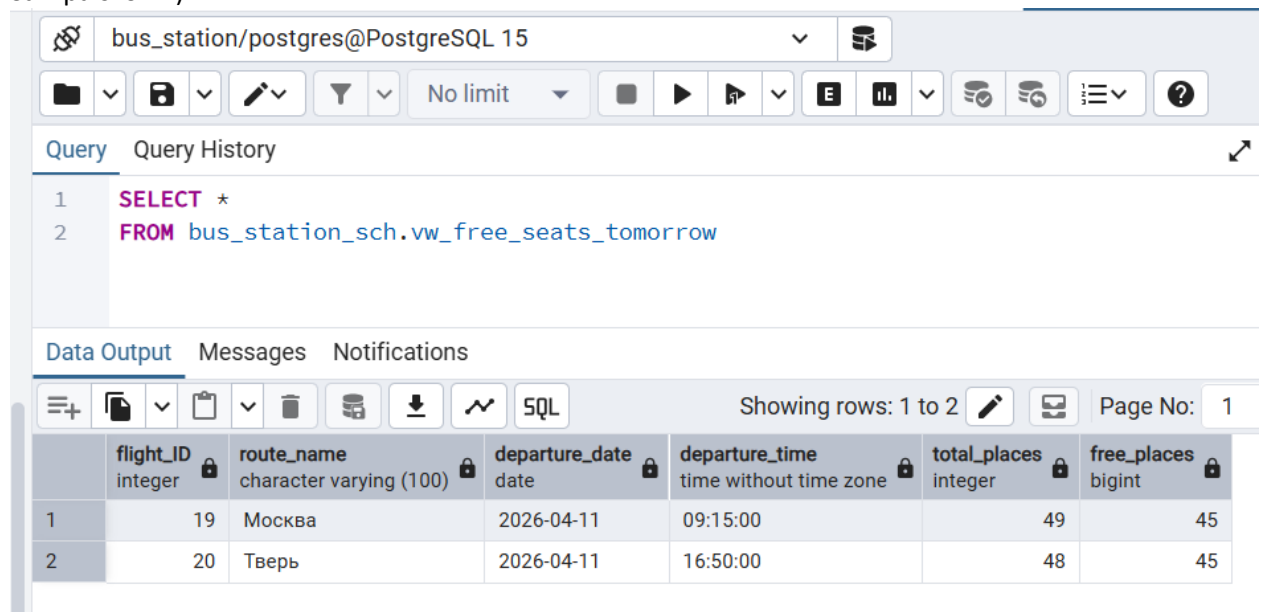
AND b.ticket_status = 'PAID'
AND b.place_number IS NOT NULL
WHERE r.departure_date = CURRENT_DATE + 1
GROUP BY
    r."flight_ID",
    p.destination_name,
    r.departure_date,
    r.departure_time,
    m.place_number;

```

Проверка:

```
SELECT *
```

FROM bus_station_sch.vw_free_seats_tomorrow (лабораторная исправлялась 10 числа поэтому на завтра это 11)



The screenshot shows a PostgreSQL query editor interface. The top bar indicates the connection is to 'bus_station/postgres@PostgreSQL 15'. Below the toolbar, the 'Query' tab is active, showing a SQL query. The 'Data Output' tab is also visible, displaying the results of the query in a table format. The table has 7 columns: flight_ID, route_name, departure_date, departure_time, total_places, and free_places. The results show two rows of data.

	flight_ID integer	route_name character varying (100)	departure_date date	departure_time time without time zone	total_places integer	free_places bigint
1	19	Москва	2026-04-11	09:15:00	49	45
2	20	Тверь	2026-04-11	16:50:00	48	45

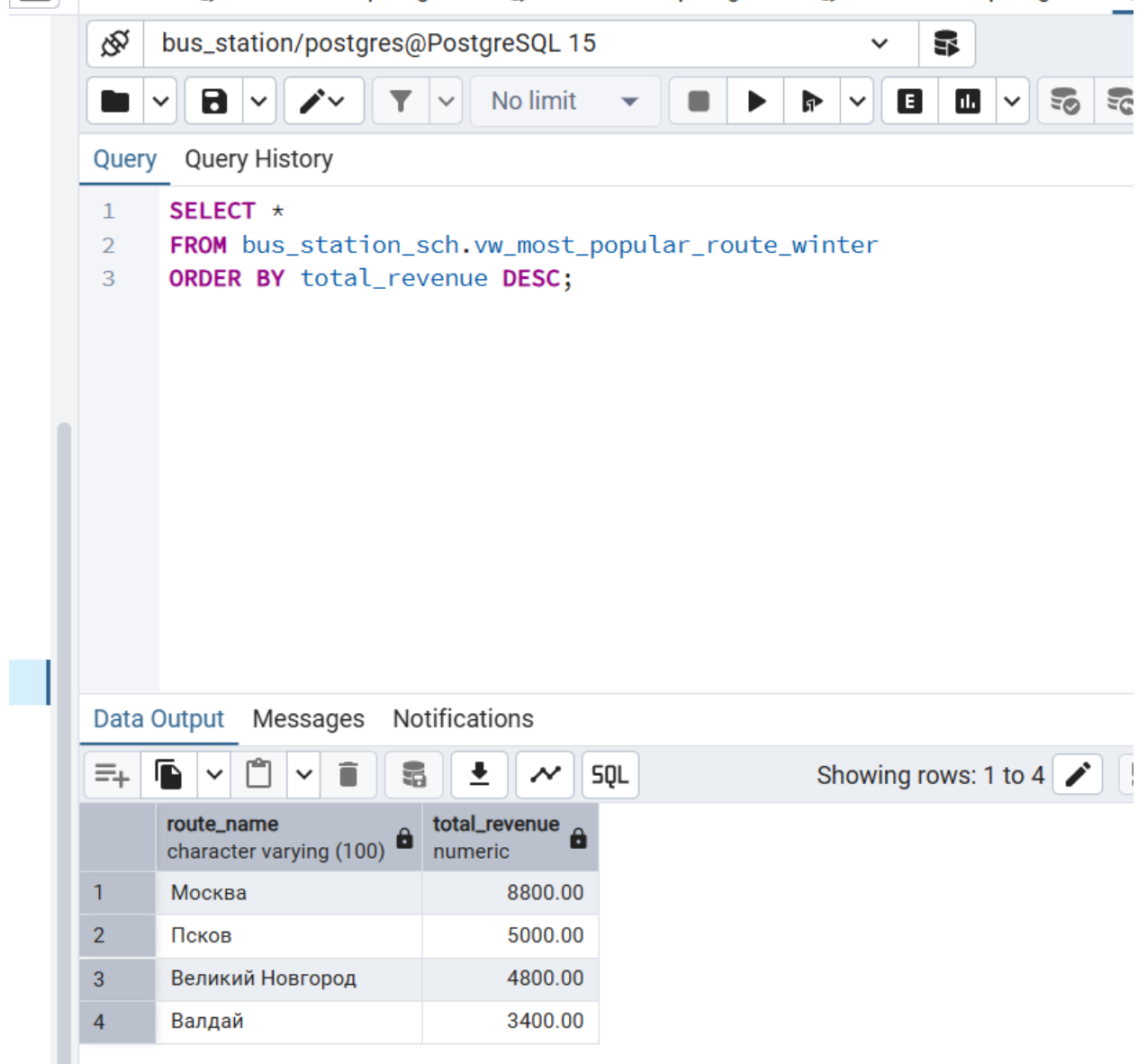
самый популярный маршрут этой зимой.

```

CREATE VIEW bus_station_sch.vw_most_popular_route_winter AS
SELECT
    p.destination_name AS route_name,
    SUM(b.ticket_price) AS total_revenue
FROM bus_station_sch.reis r
JOIN bus_station_sch.punkt_naznacheniya p
ON r."destination_ID" = p."destination_ID"
JOIN bus_station_sch.bilet b
ON r."flight_ID" = b."flight_ID"
WHERE r.departure_date >= DATE '2026-01-01'
AND r.departure_date <= DATE '2026-02-28'
AND b.ticket_status = 'PAID'
GROUP BY p.destination_name;

```


Проверка, самая первая строка - это необходимый результат:



The screenshot shows a PostgreSQL query editor interface. The top bar indicates the connection is to 'bus_station/postgres@PostgreSQL 15'. Below the toolbar, the 'Query' tab is active, displaying the following SQL query:

```
1 SELECT *
2 FROM bus_station_sch.vw_most_popular_route_winter
3 ORDER BY total_revenue DESC;
```

The 'Data Output' tab is also visible, showing the results of the query. The results are displayed in a table with two columns: 'route_name' (character varying (100)) and 'total_revenue' (numeric). The table shows 4 rows of data, sorted by total revenue in descending order.

	route_name	total_revenue
1	Москва	8800.00
2	Псков	5000.00
3	Великий Новгород	4800.00
4	Валдай	3400.00

3. Запросы на модификацию данных с подзапросами INSERT

Ситуация: пассажир, который ранее был в базе, хочет, чтобы ему оформили билет на тот же рейс, на который едет его друг, но он хочет купить билет без места.

Какая логика у вставки:

Беру максимально существующий номер билета и прибавляю 1 – это я создаю новый билет.

Беру билет без места

Дату покупки взяла равной дате отправления рейса, это удовлетворяет условию, что билет куплен не позже даты отправления, точно не раньше, чем за 10 дней.

Цена: нахожу рейс друга, среди билетов этого рейса беру минимальную цену такого билета, который при этом имеет номер места не NULL, умножаю на 0.9, чтобы получить цену билета без места

Номер рейса беру как у друга

Айдишник пассажира беру уже существующего в базе

Беру те же остановки, что и у друга

А также ставлю статус Оплачено

```
INSERT INTO bus_station_sch.bilet
```

```
(
```

```
    "ticket_ID",
```

```
    place_number,
```

```
    purchase_date,
```

```
    ticket_price,
```

```
    purchase_method,
```

```
    "flight_ID",
```

```
    "ID_passenger",
```

```
    "stop_ID",
```

```
    ticket_status
```

```
)
```

```
VALUES
```

```
(
```

```
    (SELECT MAX(b."ticket_ID") + 1
```

```
      FROM bus_station_sch.bilet b),
```

```
    NULL,
```

```
    (
```

```
      SELECT r.departure_date
```

```
      FROM bus_station_sch.reis r
```

```
      WHERE r."flight_ID" =
```

```
        (
```

```
          SELECT b2."flight_ID"
```

```
          FROM bus_station_sch.bilet b2
```

```
          WHERE b2."ticket_ID" = 10
```

```
        )
```

```
    ),
```

```
    (
```

```
      SELECT MIN(b1.ticket_price) * 0.9
```

```

FROM bus_station_sch.bilet b1
WHERE b1."flight_ID" =
(
    SELECT b2."flight_ID"
    FROM bus_station_sch.bilet b2
    WHERE b2."ticket_ID" = 10
)
AND b1.place_number IS NOT NULL
AND b1.ticket_status = 'PAID'
),
'0',
(
    SELECT b2."flight_ID"
    FROM bus_station_sch.bilet b2
    WHERE b2."ticket_ID" = 10
),
5,
(
    SELECT b2."stop_ID"
    FROM bus_station_sch.bilet b2
    WHERE b2."ticket_ID" = 10
),
'PAID'
);

```

bus_station/postgres@PostgreSQL 15

Query Query History

```

1  INSERT INTO bus_station_sch.bilet
2  (
3      "ticket_ID",
4      place_number,
5      purchase_date,
6      ticket_price,
7      purchase_method,
8      "flight_ID",
9      "ID_passenger",
10     "stop_ID",
11     ticket_status
12 )
13 VALUES
14 (
15     (SELECT MAX(b."ticket_ID") + 1
16      FROM bus_station_sch.bilet b),
17     NULL,
18     (
19         SELECT r.departure_date
20         FROM bus_station_sch.reis r
21         WHERE r."flight_ID" =
22             (
23                 SELECT b2."flight_ID"
24                 FROM bus_station_sch.bilet b2
25                 WHERE b2."ticket_ID" = 10

```

Data Output Messages Notifications

INSERT 0 1

Query returned successfully in 239 msec.

Билет успешно встался:

bus_station/postgres@PostgreSQL 15

Query Query History Scratch Pad x

```

1  SELECT *
2  FROM bus_station_sch.bilet
3  ORDER BY "ticket_ID" DESC;

```

Data Output Messages Notifications

Showing rows: 1 to 83 Page No: 1 of 1

ticket_ID [PK] integer	place_number integer	purchase_date date	ticket_price numeric	purchase_method character (1)	flight_ID integer	ID_passenger integer	stop_ID integer	ticket_status character varying (15)
83	[null]	2026-04-10	1440.000	0	3	5	9	PAID

4. Запрос на модификацию 2. UPDATE с подзапросом

Ситуация: пассажир уже купил билет, но ему понадобилось доплатить за багаж, поэтому стоимость билета увеличивается

Данные до обновления:

bus_station/postgres@PostgreSQL 15

Query Query History

```

1  SELECT билет."ticket_ID",
2      билет."ID_passenger",
3      билет."flight_ID",
4      билет.ticket_price,
5      билет.ticket_status
6  FROM bus_station_sch.билет
7  WHERE билет."ID_passenger" = 5
8        AND билет."flight_ID" =
9        (
10         SELECT reis."flight_ID"
11         FROM bus_station_sch.reis
12         JOIN bus_station_sch.punkt_naznacheniya
13           ON reis."destination_ID" = пункт_naznacheniya."destination_ID"
14         WHERE reis.departure_date = DATE '2026-04-10'
15               AND reis.departure_time = TIME '10:45'
16               AND пункт_naznacheniya.destination_name = 'Тверь'
17       );

```

Data Output Messages Notifications

Showing rows: 1 to 1

	ticket_ID [PK] integer	ID_passenger integer	flight_ID integer	ticket_price numeric	ticket_status character varying (15)
1	5	5	2	1400.00	PAID

```

UPDATE bus_station_sch.билет
SET ticket_price = ticket_price + 300
WHERE "ID_passenger" = 5
      AND "flight_ID" =
      (
        SELECT reis."flight_ID"
        FROM bus_station_sch.reis
        JOIN bus_station_sch.punkt_naznacheniya
          ON reis."destination_ID" = пункт_naznacheniya."destination_ID"
        WHERE reis.departure_date = DATE '2026-04-10'
              AND reis.departure_time = TIME '10:45'
              AND пункт_naznacheniya.destination_name = 'Тверь'
      )
AND ticket_status = 'PAID';

```

bus_station/postgres@PostgreSQL 15

Query Query History

```

1  UPDATE bus_station_sch.bilet
2  SET ticket_price = ticket_price + 300
3  WHERE "ID_passenger" = 5
4     AND "flight_ID" =
5     (
6         SELECT reis."flight_ID"
7         FROM bus_station_sch.reis
8         JOIN bus_station_sch.punkt_naznacheniya
9             ON reis."destination_ID" = punkt_naznacheniya."destination_ID"
10        WHERE reis.departure_date = DATE '2026-04-10'
11              AND reis.departure_time = TIME '10:45'
12              AND punkt_naznacheniya.destination_name = 'Тверь'
13    )
14    AND ticket_status = 'PAID';
15

```

Data Output Messages Notifications

UPDATE 1

Query returned successfully in 107 msec.

Проверка, что данные обновились:

bus_station/postgres@PostgreSQL 15

Query Query History

```

1  SELECT bilet."ticket_ID",
2         bilet."ID_passenger",
3         bilet."flight_ID",
4         bilet.ticket_price,
5         bilet.ticket_status
6  FROM bus_station_sch.bilet
7  WHERE bilet."ID_passenger" = 5
8     AND bilet."flight_ID" =
9     (
10        SELECT reis."flight_ID"
11        FROM bus_station_sch.reis
12        JOIN bus_station_sch.punkt_naznacheniya
13            ON reis."destination_ID" = punkt_naznacheniya."destination_ID"
14        WHERE reis.departure_date = DATE '2026-04-10'
15              AND reis.departure_time = TIME '10:45'
16              AND punkt_naznacheniya.destination_name = 'Тверь'
17    );

```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1

	ticket_ID [PK] integer	ID_passenger integer	flight_ID integer	ticket_price numeric	ticket_status character varying (15)
1	5	5	2	1700.00	PAID

5. Запрос на модификацию 3. DELETE с подзапросом

Ситуация: пассажир отменил поездку на конкретный рейс.

Значит нужно удалить билет пассажира на рейс в конкретный пункт назначения на указанную дату (Пассажир и рейс определяются подзапросами)

```
DELETE FROM bus_station_sch.bilet
WHERE "ID_passenger" =
(
    SELECT p."ID_passenger"
    FROM bus_station_sch.passazhir p
    WHERE p."ID_passport" = 1
)
AND "flight_ID" =
(
    SELECT r."flight_ID"
    FROM bus_station_sch.reis r
    JOIN bus_station_sch.punkt_naznacheniya pn
        ON r."destination_ID" = pn."destination_ID"
    WHERE r.departure_date = DATE '2026-04-10'
        AND pn.destination_name = 'Москва'
        AND r.departure_time = TIME '08:30'
);
```

После удаления:

The screenshot shows a PostgreSQL query editor interface. The top bar indicates the connection is to 'bus_station/postgres@PostgreSQL 15'. Below the toolbar, the 'Query' tab is active, displaying a SQL statement that deletes tickets from the 'bilet' table based on subqueries for passenger ID and flight ID. The bottom panel shows the 'Data Output' tab with a table schema for 'bilet'.

```
1 SELECT *
2 FROM bus_station_sch.bilet
3 WHERE "ID_passenger" =
4 (
5     SELECT p."ID_passenger"
6     FROM bus_station_sch.passazhir p
7     WHERE p."ID_passport" = 1
8 )
9 AND "flight_ID" =
10 (
11     SELECT r."flight_ID"
12     FROM bus_station_sch.reis r
13     JOIN bus_station_sch.punkt_naznacheniya pn
14         ON r."destination_ID" = pn."destination_ID"
15     WHERE r.departure_date = DATE '2026-04-10'
16         AND pn.destination_name = 'Москва'
17         AND r.departure_time = TIME '08:30'
18 );
```

ticket_ID	place_number	purchase_date	ticket_price	purchase_method	flight_ID	ID_passenger	stop_ID	ticket_status
[PK] integer	integer	date	numeric	character (1)	integer	integer	integer	character varying (15)

6. План выполнения запроса EXPLAIN:

Простой индекс:

Сотрудник ищет конкретный билет по его номеру, например для проверки возврата или изменения статуса.

Без индекса:

The screenshot shows a PostgreSQL query editor interface. The query being executed is:

```
1 EXPLAIN ANALYZE
2 SELECT *
3 FROM bus_station_sch.bilet_index_test
4 WHERE "ticket_ID" = 1001234;
```

The results are displayed in the 'Data Output' tab, showing the 'QUERY PLAN' for the query. The plan indicates a sequential scan on the 'bilet_index_test' table, which is not indexed on the 'ticket_ID' column.

	QUERY PLAN
1	Seq Scan on bilet_index_test (cost=0.00..44.03 rows=1 width=36) (actual time=0.156..0.229 rows=1 loop...
2	Filter: ("ticket_ID" = 1001234)
3	Rows Removed by Filter: 2081
4	Planning Time: 0.109 ms
5	Execution Time: 0.254 ms

Индекс:

```
CREATE INDEX idx_bilet_test_ticket_id
ON bus_station_sch.bilet_index_test("ticket_ID");
```

С индексом:

bus_station/postgres@PostgreSQL 15

Query Query History Scratch Pad x

```

1  EXPLAIN ANALYZE
2  SELECT *
3  FROM bus_station_sch.bilet_index_test
4  WHERE "ticket_ID" = 1001234;

```

Data Output Messages Notifications

Showing rows: 1 to 4 Page No: 1 of 1

QUERY PLAN	
	text
1	Index Scan using idx_bilet_test_ticket_id on bilet_index_test (cost=0.28..8.30 rows=1 width=36) (actual time=0.054..0.055 rows=1 loo...
2	Index Cond: ("ticket_ID" = 1001234)
3	Planning Time: 0.492 ms
4	Execution Time: 0.080 ms

Время сократилось и видно, что индекс используется.

Составной индекс:

найти оплаченные онлайн билеты на конкретный рейс с определенной ценой, нужно для проверки продаж или поиск билетов подсчет их по параметрам оплаты.

Без индекса:

bus_station/postgres@PostgreSQL 15

Query Query History Scratch Pad

```

1  EXPLAIN ANALYZE
2  SELECT *
3  FROM bus_station_sch.bilet_index_test
4  WHERE "flight_ID" =
5  (
6      SELECT "flight_ID"
7      FROM bus_station_sch.bilet
8      WHERE "ticket_ID" =
9      (
10         SELECT MIN("ticket_ID")
11         FROM bus_station_sch.bilet
12     )
13 )
14 AND ticket_status = 'PAID'
15 AND ticket_price = 1234;

```

Data Output Messages Notifications

Showing rows: 1 to 16 Page No: 1 of 1

QUERY PLAN
text
1 Seq Scan on bilet_index_test (cost=2.34..56.78 rows=1 width=36) (actual time=0.126..0.446 rows=4 loops=1)
2 Filter: (("flight_ID" = \$2) AND ((ticket_status)::text = 'PAID'::text) AND (ticket_price = '1234'::numeric))
3 Rows Removed by Filter: 2078
4 InitPlan 3 (returns \$2)

Planning Time: 0.306 ms

Execution Time: 0.482 ms

Индекс:

```

CREATE INDEX idx_bilet_test_flight_status_price
ON bus_station_sch.bilet_index_test("flight_ID", ticket_status,
ticket_price);

```

После индекса:

bus_station/postgres@PostgreSQL 15

Query Query History Scratch Pad

```

1  EXPLAIN ANALYZE
2  SELECT *
3  FROM bus_station_sch.bilet_index_test
4  WHERE "flight_ID" =
5  (
6      SELECT "flight_ID"
7      FROM bus_station_sch.bilet
8      WHERE "ticket_ID" =
9      (
10         SELECT MIN("ticket_ID")
11         FROM bus_station_sch.bilet
12     )
13 )
14 AND ticket_status = 'PAID'
15 AND ticket_price = 1234;

```

Data Output Messages Notifications

Showing rows: 1 to 15 Page No: 1 of 1

QUERY PLAN
1 Index Scan using idx_bilet_test_flight_status_price on bilet_index_test (cost=2.62..10.64 rows=1 width=36) (actual time=0.098..0.105 rows=1)
2 Index Cond: (("flight_ID" = \$2) AND ((ticket_status)::text = 'PAID'::text) AND (ticket_price = '1234'::numeric))
3 InitPlan 3 (returns \$2)
4 -> Seq Scan on bilet bilet_1 (cost=0.32..2.34 rows=1 width=4) (actual time=0.037..0.045 rows=1 loops=1)
5 Filter ("ticket_ID" = \$1)

Total rows: 15 Query complete 00:00:00.112 CRLF Ln 15, Col 25

Planning Time: 2.173 ms

Execution Time: 0.148 ms

Индекс используется. Время сократилось.

Вывод:

В ходе выполнения лабораторной работы были изучены запросы на выборку и модификацию данных в PostgreSQL, а также средства создания представлений и индексов в pgAdmin 4. Для базы данных «Автовокзал» были выполнены запросы SELECT, созданы представления VIEW, составлены запросы INSERT, UPDATE и DELETE с использованием подзапросов. Также были изучены история запросов, сообщения сервера и план выполнения SQL-команд с помощью EXPLAIN ANALYZE. В работе были созданы простой и составной индексы, выполнено сравнение времени выполнения запросов до и после индексирования, что позволило оценить влияние индексов на производительность запросов. В результате работы были получены практические навыки анализа, модификации и оптимизации запросов к базе данных PostgreSQL.